Florida Mental Health App

Helping people in Florida find a mental health provider who takes their insurance and is actually taking new patients.

Every listing carries one of these

ACCEPTING

WAITLIST

AT CAPACITY

Live, per provider. Not a phone call to find out.

About Me

I come from a background in Tech.
Attended Erwin Technical college twice, once for Networking and Wireless Telecommunications where I was the top performer in my class. The other I was accelerated through the Cloud Computing program and selected early for the PeopleShores apprenticeship. I have been a tech in many computer and cell phone repair shops as well as doing hardware and systems administration in the United States Marine Corps.



Most directories answer the wrong question

They tell you who exists. That was never the barrier.

67

counties
with seven regional groups running services, urban and rural

Who takes my insurance?

Filter by plan, not by guesswork.

Who is actually accepting?

We track open spots, and the number updates every time a referral is decided.

How long will I wait?

Usual wait time, and how many people are ahead of you.

Search

Four filters. Use any of them, or none at all.

- **County** — all 67, pulled from the database
- **Insurance plan** — 16 plans across six types
- **Telehealth** — offered or not
- **Availability** — only the ones taking new patients

Twenty results a page. Change a filter and you go back to page one, so you can never end up on page three of a one-page list.



One query, every combination

The moment the system is built around

A navigator works a queue of pending requests, oldest first, and resolves each one.

ACCEPTED

Capacity was there

One opening gets used up. If that was the last one, the provider stops showing as taking new patients — those two can never disagree, because the one piece of code that lowers the count also turns off the label.

WAITLISTED

The last slot went first

The request is not lost, and the provider is not overbooked.

Two navigators, one last slot

If two staff accept at the same moment one person waits.

Under the hood

Three layers, and they only talk one direction.

React

Single-page front end

9 pages, locked down by role

Spring Boot

REST API

26 endpoints, rules in the middle layer

MySQL

Relational store

11 tables, every link enforced by the database

Security

- Passwords encryption with BCrypt, never stored or sent back in any form
- You get a signed pass at login. Nothing kept on the server, so any copy of the app can serve you
- Two roles, checked in two separate places, so forgetting one is not a hole
- Who you are comes from your login pass, never from the request

Deliberate choices

- What goes to the browser is always something I designed, never a raw database row — so renaming a column breaks nothing
- Nothing is ever really deleted. A provider with referral history is switched off, not removed
- Every error comes back the same shape, from one place

How the backend is laid out

Four layers, and they only talk one direction. I will go through them inside out.

Controller
Takes requests,
hands back answers.

›

Service
The actual rules.
All-or-nothing saving.

›

Repository
Database questions.
Spring writes the code.

›

Entity
One table. Protects
its own rules.

Every class says what it is

`Provider`	the entity — one row
`ProviderRepository`	its queries
`ProviderService`	its rules
`ProviderController`	its endpoints

Pick any noun in the domain and you can guess all four file names without looking.

Every method says what it does

`findByLicenseNumber`	the name IS the query
`findByIdForUpdate`	“ForUpdate” means a lock
`canAcceptReferral()`	a question — changes nothing
`consumeSlot()`	a command — returns nothing
`transitionTo(…)`	a command that writes history

None of these have code underneath — Spring writes it from the names.

Service · where the rules live

service/ProviderService.java:60

```
@Transactional(readOnly = true)
public Page<ProviderDtos.Summary> search(
        ProviderDtos.SearchCriteria criteria, Pageable pageable) {
    return providerRepository.search(
            criteria.countyId(),
            criteria.insurancePlanId(),
            criteria.telehealth(),
            criteria.acceptingOnlyOrFalse(),
            pageable)
        .map(this::toSummary);
}
```

Where the rules live, and where the database stops

All-or-nothing saving starts here, never in the controller and never in the repository, so a piece of work is always one method. The last line is the other boundary — the database object stops here, and what goes out is a small version I built. Those small objects are named for their shape, not their table: SearchCriteria goes in, Summary is one row in a list, Response is the full detail.

Service · the one that makes a decision

service/ReferralService.java:216

```
if (requested == Status.ACCEPTED) {

    Provider provider = providerRepository
            .findByIdForUpdate(referral.getProvider().getId())
            .orElseThrow(() -> new ResourceNotFoundException(...));

    if (provider.canAcceptReferral()) {
        provider.consumeSlot();
    } else {
        applied = Status.WAITLISTED;              // not an error
        provider.addToWaitlist();
    }
}
```

> **A decision, not a write**
>
> The copy of the provider it is already holding could be out of date, so it is fetched again and held before anything gets counted. If the room is gone the method does not fail — it changes what it is doing and says WAITLISTED instead. The whole thing is all-or-nothing, so the count, the status and the history entry save together or not at all.

Repository · one query, every combination

repository/ProviderRepository.java:43

```
SELECT DISTINCT p FROM Provider p
JOIN p.organization o
LEFT JOIN p.insurancePlans ip
WHERE p.active = true
  AND (:countyId    IS NULL OR o.county.id = :countyId)
  AND (:insuranceId IS NULL OR ip.id       = :insuranceId)
  AND (:telehealth  IS NULL OR p.offersTelehealth = :telehealth)
  AND (:acceptingOnly = false OR p.acceptingNewClients = true)
```

Blank turns itself off

If a filter is empty its line is automatically true, so it drops out. One query, all sixteen combinations.

DISTINCT

Pulling in insurance plans duplicates rows. Without this, someone who takes two plans shows up twice.

Repository · the lock

repository/ProviderRepository.java:73

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT p FROM Provider p WHERE p.id = :id")
Optional<Provider> findByIdForUpdate(@Param("id") Long id);
```

entity/Provider.java:196 — the rule lives with the data

```
public void consumeSlot() {
    if (openSlots <= 0) throw new IllegalStateException(...);
    openSlots--;
    if (openSlots == 0) {
        acceptingNewClients = false;            // keep it honest
    }
}
```

> **Two fields that can never disagree**
>
> The only code that lowers the count also switches off the accepting label, so the directory cannot show “taking new patients” next to zero openings

Controller · the thin edge

Last on purpose. It is the most boring file in the project, and that is the point.

`controller/ProviderController.java — @RequestMapping("/api/providers")`

```
@GetMapping("/search")
public Page<ProviderDtos.Summary> search(
        ProviderDtos.SearchCriteria criteria,
        @PageableDefault(size = 20) Pageable pageable) {
    return providerService.search(criteria, pageable);
}
```

`… and the only other thing a controller does — identity`

```
@PostMapping("/{id}/save")
public ResponseEntity<ProviderDtos.SavedResponse> save(
        @PathVariable Long id,
        @AuthenticationPrincipal UserPrincipal principal) {
```

> **No logic, and no user id in the request**
>
> The web addresses are just the table name, and Spring reads the address and fills in the filters object itself. Who you are comes off your login pass on the last line — never the address, never the body — so there is no number to change to get at someone else's data.

Find Care Florida

A directory of mental health providers across all 67 Florida counties, with the one thing most directories leave out: whether they are taking new clients.

Sign in

Email

Password

Sign in

No account yet? **Create one**

You do not need an account to see crisis and statewide help. Go to resources.

Where it stands

180
tests

83.5%
code coverage

0
open quality issues

Said plainly

- All the provider and patient data is made up. Every clinic name starts with "Example" on purpose — putting invented availability next to a real clinic's name would be making up a record about a real place.
- The crisis resources are the exception: 988, Crisis Text Line, 211 and NAMI Florida are real, I checked them, and they are on every screen without logging in.

Things I have learned and would do if I had more time

I learned just how tedious testing is and how crucial it is to the development process. Making sure everything works and passes security tests and functionality tests gave me a couple of extra gray hairs in my beard. If I had more time, I would’ve hosted my application in AWS